



TAQOZ P2 Boot Firmware

UPDATED! January 12, 2019 ([pubdoc link](#)) ([edit/comment version](#))

QUICK LINKS - [P2 DOCUMENTS](#) + [MY P2 FOLDER](#) (everything)

INTRODUCTION

Back in 2012 I decided to write Tachyon Forth for the Parallax Propeller P8X32A or as we refer to it now simply as P1. Briefly, the reason for writing a Forth was not for Forth's sake although the appeal there was an interactive development environment resident on the P1, it was to get more out of the limited memory of the P1, but to also do so as fast as was possible for what is essentially a virtual machine with the view that serious commercial products could be produced based on the flexible P1.

In the years that followed Tachyon has proved itself, through the various original bytecode versions through to the version that now uses 16-bit word codes. The successor to the P1 is simply referred to as the P2 and although it is only available at present as an FPGA image for testing and development, nonetheless serious testing has been carried out for many years. As part of this testing I ported Tachyon across for the P2 and now Chip has announced that the P2 to be available hopefully sometime in 2018 will have 16kB of ROM of which he would only be using a couple of kB or so for the boot loader etc.

TAQOZ is implemented primarily for debugging hardware or testing out "what ifs" but it is still nonetheless a full Forth custom designed for the P2 and its hardware. As it stands the native TAQOZ wordcode can address code in a 16-bit address space while dictionary and data may reside anywhere in memory. So having software written in TWC (TAQOZ Word Code) is very compact and fast and 60kB of code represents a very very large program whereas FAT32, Ethernet servers and VGA including the Tachyon kernel and extensions would all fit in around 20kB of code space on the P1.

TAQOZ provides an interactive hardware debugging environment through its Forth console via a serial terminal emulator. Even without any "software" it is possible to exercise the hardware and write code on top of the TAQOZ dictionary, thus extending the language's vocabulary of words, all of which are a combination of various functions, constants, variables etc that are referred to by name. Parameters though are passed postfix style so that data is pushed onto a data stack and precede functions.

GETTING THE TAQOZ ROM PROMPT UP

Connect a terminal and I recommend using ones that don't drop characters and support ANSI such as TeraTerm, RealTerm, GtTerm, Minicom. You can use any baud rate from 9600 to 2M baud (or more) but by default select 115200, 8 data, 1 stop, no parity, no handshakes, no echo, no extra CR/LFs etc.

Now that you have your P2 connected to a terminal you can type a "greater than sign" and a "space" to autobaud the serial loader after which you can hit "escape" to escape to TAQOZ. That is the ASCII key sequence for \$3E,\$20,\$1B.

GETTING STARTED

If all is well you should have the splash logo and the prompt.

```
Cold start
-----
Parallax P2  .:.-TAQOZ-:.. V1.1--v33h      190219-1900
-----
TAQOZ#
```

Now enter WORDS to see which words are in the dictionary (or hit ^W)

TAQOZ - Tachyon P2 Forth in ROM

Updated automatically every 5 minutes

```

U// /*/ UM// C++ C-- W++ W-- ++ -- RND GETRND SQRT SETDACS ~~~
W~ W~~
C~ C~~ L>S>W L>W W>B W>L B>W B>L MINS MAXS MIN MAX ABS -
NEGATE ?NEGATE
NEGATE ON TRUE -1 FALSE OFF GOTO IF ELSE THEN BEGIN UNTIL
AGAIN WHILE REPEAT
SWITCH CASE@ CASE= CASE> BREAK CASE ADO DO LOOP +LOOP
FOR NEXT ?NEXT I
J LEAVE IC@ I+ BOUNDS H L T F R HIGH LOW FLOAT PIN@ WRPIN
WXPIN WYPIN RDPIN
RQPIN AKPIN WAITPIN WRACK PIN @PIN ns PW PULSE PULSES HILO
DUTY NCO HZ
KHZ MHZ MUTE BLINK PWM SAW BIT BAUD TXD RXD TXDAT WAITX
WAITCNT REBOOT
RESET ?EXIT EXIT NOP CALL JUMP >R R> >L L> !SP DEPTH COG@
COG! LUT@ LUT!
COGID COGINIT COGSTOP NEWCOG COGATN POLLATN SETEDG
POLLEDG KEY WKEY KEY!
CON NONE COM CONKEY CONEMIT SEROUT EMIT EMITS CRLF CR
CLS SPACE SPACES
RAM DUMP: DUMP DUMPW DUMPL DUMPA DUMPAW QD QW DEBUG
lsio COG LUT KB MB
M . PRINT .AS .AS" .DECL .DEC4 HOLD #> <# # #S <D> U. .DEC .BIN .H .B
.BYTE
.W .WORD .L .LONG .ADDR PRINT$ LENS$ " ." CTYPE ?EXIT DATA?
ERASE FILL CMOVE
<CMOVE s ms us CNT@ LAP LAP@ .LAP .ms HEX DEC BIN .S WORDS
@WORDS GET$
SEARCH $># @DATA HERE @HERE @CODES uemit ukey char delim names
TASK REG
@WORD SPIN || , [W] ["] NULL$ $! $= ASM FORGET CREAT$ CREATE
VAR pub
pri pre : ; [ ] ' := ==! ALIGN DATCON ALLOT org bytes words longs byte
word long res [C] GRAB NFA' CPA CFA \ --- ( { } IFNDEF IFDEF TAQOZ
TERM
AUTO SPIRD SPIRDL SPIWB SPICE SPIWC SPIWW SPIWM SPIWL
SPIPINS SPIRX SPITXE
SPITX WAIT CLKDIV RCSLOW HUBSET WP WE CLKHZ ERROR SFPINS
SF? SFWE SFINS
SFWD SFSID SFJID SFER4 SFER32 SFER64 SFERASE SFWRPG BACKUP
RESTORE SFRDS
SFWRS SFC@ SFW@ SF@ SF .SF SDBUF sdpins MOUNT DIR !SD !SX SD?
CMD ACMD
cid SDWR SDRDS SDWRS FLUSH FOPEN FLOAD FGET FREAD FWRITE
SECTOR SDRD SDRDS
SDADR SD@ SD! SDC@ SDC! SDW@ SD @FAT @BOOT @ROOT fat END
432 ok
TAQOZ#

```

These are all the words that TAQOZ understands that you can type in and run or build into new words that are added to the dictionary. Let's just try interacting with TAQOZ for a moment and make an LED blink. Assume we have an LED connected to P5 we can make it blink very simply with **5 BLINK** which is defined simply as **PIN 2 HZ** where the word PIN takes the argument 5 off the top of the data stack and HZ takes the 2 and configures the selected pin as an NCO with a frequency of 2 HZ. Since TAQOZ boots up in RCFAST it is running at a 25MHz speed rather than the 80MHz that was used in the FPGA version, so it will be slower. To make it blink faster and since the pin has already been selected through PIN we can just type in **10 HZ** instead of any other value.

Remember that all words and numbers should be separated by at least one space but they can even be on separate lines.

```

TAQOZ# 5 BLINK ok
TAQOZ# 10 HZ ok

```

The HZ word is using the smartpin mode so it doesn't need any code running continually to make it blink but to make the smartpin stop type **MUTE**. If we have multiple LEDs connected to P0 to P7 we can make them all blink in one simple loop **8 0 DO I BLINK LOOP** which specifies a limit and a starting index used by DO to setup a loop consisting of **I BLINK**. The I leaves the current loop index value on the stack which is then used by BLINK so that each time through the loop the index will increment up to the limit of 8 and exit the loop. This means that the index I will go through the values 0,1,2,3,4,5,6,7 branching back each time to just after DO but once it reaches 8 it does not loop back but continues after the word LOOP which in the case of the example executes an automatic EXIT or return operation back to the console ready for more input.

You can customise this simple 8 LED blinker by having a different blink rate for each LED like this: **8 0 DO I PIN I 2 + HZ LOOP**. Now that creates an interesting pattern with the interaction between the LEDs.

```

TAQOZ# MUTE ok
TAQOZ# 8 0 DO I PIN I 2 + HZ LOOP ok

```

TAQOZ - Tachyon P2 Forth in ROM

Updated automatically every 5 minutes

BLINK	(pin --)	Select the smartpin and make it blink slowly
PIN	(pin --)	Select a smartpin to use
HZ	(n --)	Configure the preselected smart pin as an NCO set to n Hz
MUTE		Mute or stop the preselected smartpin
DO	(limit start --)	Push limit and start index and loop start address onto the loop stack.
LOOP		Increment the loop index and loop back while index <> limit
I	(-- index)	Push the current loop index value onto the stack

STACK BASICS

You may find Forth style RPN "back-to-front" from conventional languages but that doesn't mean either one is better. Do you know that we use RPN in our speech? Everyone knows it takes 60 seconds to make 1 minute. Wait a minute, where's the RPN? Well in most computer language "styles" we might express that as "minute(1) = seconds(60)" so the latter is more mathematical but the former is more natural for English speakers. Where it doesn't seem natural is when we have more than one parameter as in adding numbers. Try this: **8 4 +** which performs the addition after it has the parameters with the result left on the stack. After all though, you can't bake a cake before you have both eggs and flour available, just like Forth parameters where we bake the 8 and 4 with a + operation. You can type a single tiny dot . to print (and use) the number off the stack or you can type **.S** to view the stack.

```
TAQOZ# 8 4 ok
TAQOZ# .S
DATA STACK (2)
1 $0000.0004 4
2 $0000.0008 8 ok
TAQOZ# + ok
TAQOZ# .S
DATA STACK (1)
1 $0000.000C 12 ok
TAQOZ#
```

NEW WORDS

WORD	STACK	DESCRIPTION
+	{ n1 n2 -- n3 }	Add (and use) n1 and n2 and push the result n3 back onto the data stack
.S		Print and view the contents of the data stack non-destructively
.	(n --)	Print the signed value in the current number base (default decimal)

BLINKING LEDs AGAIN

Making the LED blink the hard old way we need to stop those blinking LEDs first but they won't stop blinking after a reset or even a cold start (type ^Z).

We could power it back up or we could use Forth to do the work for us: **8 0 DO I PIN MUTE LOOP**

Now that we shut them up here's the old way in Forth, but it is only a single line: **BEGIN 5 HIGH 100 ms 5 LOW 100 ms AGAIN**

That works but now it is locked up in an endless loop bounded by BEGIN AGAIN and the console doesn't respond!!! Don't panic, just hit escape four times in a row to get back to the console. If I wanted any key to stop the looping I could have typed: **BEGIN 5 HIGH 100 ms 5 LOW 100 ms KEY UNTIL**

KEY returns a character from the serial buffer and pushes the value onto the stack just like any other value. When the buffer is empty KEY returns a null value or the value 0. UNTIL uses a value off the top of the stack as a true/false flag and will continue looping until the value is true. In most languages 0 is false but any non-zero value can be true and such is the case with our BEGIN UNTIL loop.

NEW WORDS

WORD	STACK	DESCRIPTION
BEGIN		Mark (no code) the beginning of a BEGIN loop (no indexing or automatic count)

TAQOZ - Tachyon P2 Forth in ROM

Updated automatically every 5 minutes

		direction register)
ms	(n --)	Wait for n milliseconds
KEY	(-- char)	Return with the next character in the console buffer or else a value of \$00

DEFINING NEW WORDS

Forth is like tofu, it takes on the flavor of what you add to it and so the language can be extended with new definitions/functions referred to as words which are maintained in a dictionary. The dictionary is the goto place for when you are typing into the console and if what you type is not a number it will search the dictionary for a match and when found it looks up the code address field in that entry and compiles this, either as an interactive temporary compilation or as part of a new word being defined. Note though that some words are never normally compiled as they are directives needed to control the compiler and to create structures. These are referred to as immediate words (but even immediate words can be forced to compile using other immediate words!)

OVER

Now let's take the bit of code we used to create a blinking LED in software and create a new word called "BLINKY" like this:

: BLINKY BEGIN 5 HIGH 100 ms 5 LOW 100 ms KEY UNTIL ;

Remember to use at least one space between ALL words including : and ;

The name BLINKY has been added to the dictionary (you can type WORDS to check if you like) and so if we type BLINKY it will do just so.

```
TAQOZ# : BLINKY BEGIN 5 HIGH 100 ms 5 LOW 100 ms KEY UNTIL ; ok
TAQOZ# BLINKY
```

Now BLINKY is "hard wired" to P5 and 100 ms delay but we could define a better BLINKY that takes a pin number and a delay value like this:

: BLINKER (pin ms --) BEGIN OVER HIGH DUP ms OVER LOW DUP ms KEY UNTIL 2DROP ;

Welcome to manipulating the stack using DUP OVER and 2DROP (which is simply DROP DROP) etc.

```
TAQOZ# : BLINKER ( pin ms -- ) BEGIN OVER HIGH DUP ms OVER LOW
DUP ms KEY UNTIL 2DROP ; ok
TAQOZ# 2 20 BLINKER
```

If you want to run code in another cog then is a very quick and simple example to run it in COG#1 as a task. First however we need to run a new instance of TAQOZ in another cog (originally they were all loaded but the latest version doesn't automatically load them).

In this example we will choose cog#1 using **1 NEWCOG**

```
TAQOZ# 1 NEWCOG ok
```

Now that COG#1 has been loaded with a TAQOZ kernel it is sitting in a IDLE loop waiting for a task to perform. All we need to do is to give it the address of that task. To make BLINKER run continuously the blinking code is placed in a BEGIN AGAIN structure which will loop unconditionally after which we can find the address of BLINKER (using the tick symbol \$27) and store it where cog#1 will be checking in its task variable.

```
TAQOZ# : BLINKER BEGIN 7 HIGH 100 ms 7 LOW 100 ms AGAIN ; ok
TAQOZ# ' BLINKER 1 TASK W! ok
```

The last line here I would pronounce as "address of - BLINKER - 1 TASK - W STORE"

NEW WORDS

WORD	STACK	DESCRIPTION
:		Read in the following word and create a new word in the dictionary Does not execute any code while defining
;		Compile an EXIT word and ends definition - does not execute
EXIT		Exit word by returning to implicit caller
OVER	(n1 n2 -- n1 n2 n1)	Duplicate the second stack item, that is push a copy over the top.
DUP	(n1 -- n1 n1)	Duplicate the top stack item (copies and pushes)
DROP	(n1 --)	Drop the top stack item (pops and discards)
2DROP	(n1 n2 --)	Drop 2 stack items (pops the stack twice)

TAQOZ - Tachyon P2 Forth in ROM

Updated automatically every 5 minutes

TASK	(cog# -- address)	Index the cog's task variable and return with its address.
-------------	---------------------	--

Q: What code is compiled when we define a new word?

In the example of blinker we can look at the code compiled by finding the code address using the tick word which will leave the CFA (code field address) on the stack from which we can use that address and dump it in a suitable format:

```
TAQOZ# ' BLINKER $20 DUMPW
00.102C: 0074 0175 0071 D7C8 0074 0177 0071 D7C8    t.u.q...t.w.q...
00.103C: E0DA FC8A 0066 0061 013E 102C FE20 D892    ....f.a.>.,, ... ok
```

Substituting names for wordcodes

```
00.102C: OVER HIGH DUP ms OVER LOW DUP ms    t.u.q...t.w.q...
00.103C: KEY UNTIL 2DROP EXIT 013E 102C FE20 D892    ....f.a.>.,, ... ok
```

UPDATE - using the full version of TAQOZ which includes a decompiler we can see how it is constructed, although the addresses will be different:

```
TAQOZ# SEE BLINKER
1BC1B: pub BLINKER
0ABB4: 1807    7
0ABB6: 0173    HIGH
0ABB8: 1864    100
0ABBA: 0FB4    ms
0ABBC: 1807    7
0ABBE: 0175    LOW
0ABCO: 1864    100
0ABC2: 0FB4    ms
0ABC4: ABB5    BLINKER ;
      ( 18 bytes )
```

16-bit word size was chosen with DUMPW for 32 bytes and the first wordcode is \$0074 which is the cog address of OVER. In fact any wordcode that is less than \$400 is a cog/lut address which is executed by the wordcode interpreter loop. But \$D7C8 is not a cog address but points to more wordcode. The \$0074 following it is the same as the first wordcode which is OVER. We can use the tick symbol word to find the CFA of OVER to confirm this:

```
TAQOZ# ' OVER .W $0074 ok
```

Therefore, each 16-bit wordcode has a direct correspondence with a word in the BLINKER definition. If you follow it through past 2DROP (\$0066) you will find that \$0061 is not the ; word used to complete the definition, it is in fact the wordcode for EXIT since ; is an immediate word that tells the compiler to complete the definition and compile an EXIT. Wordcodes such as FC8A are compact wordcodes that are decoded further and in this case \$FC8x means conditional branch backwards by n words. So \$0A = to 10 words back from the next instruction which brings us back to the beginning.

Now as for the name BLINKER itself it is kept in the dictionary which is made up of a contiguous list of names that include the count byte and attributes, and the wordcode for the name. To find that entry use NFA' instead.

```
TAQOZ# NFA' OVER $10 DUMP
00.B406: 04 4F 56 45 52 74 00 04 53 57 41 50 7B 00 03
52      .OVERT..SWAP{..R ok
```

MORE SMARTPIN FUN

PWM is used everywhere from power supplies to motor speed control and even generating analog waveforms. By varying the on time of a repetitive pulse to the off time we can vary the average voltage (thereby power) which may be eventually filtered by mechanical motion, thermal response, or by a filter of some kind. The ratio of on to off time is called the duty cycle meaning that a square wave that has equally on and off has 50% duty cycle and can only deliver an average of 50% of the voltage whereas an on time of 1 in 4 would deliver an average of 25%. If you connect a heating element via a suitable driver you can vary the amount of heat simply by controlling the duty cycle and if the repetitive frequency is high enough the thermal response "filters" these pulses. Slow it right down from hundreds or thousands of times per second to many seconds and those pulses of heat will no longer be filtered or averaged out. Similarly we can generate a voltage by filtering a PWM signal with a simple resistor and capacitor filter so that 0% duty outputs 0V and 100% duty outputs the full signal voltage (normally 3.3V) but 25% will produce 825mV. PWM is used in place of linear control because it can be controlled digitally and without the losses that result from "restricting" the power, a bit like putting your hand on an electric motor to try and slow it down!!.

The smartpins support a few different modes of PWM but if we have an LED

TAQOZ - Tachyon P2 Forth in ROM

Updated automatically every 5 minutes

expressed as a percentage. The longer the frame time the greater the resolution, but also the lower the repetitive frequency which is also prescaled by the last parameter.

Try different values but lastly try making the frequency value very high for a very slow frequency, slow enough that POV filtering becomes ineffective. Try **10 100 10000 PWM** and you will see the LED pulse and now it seems brighter even though the duty cycle hasn't changed. Drop back to 1% with **1 100 10000 PWM** and it will still flicker the same rate but be much dimmer.

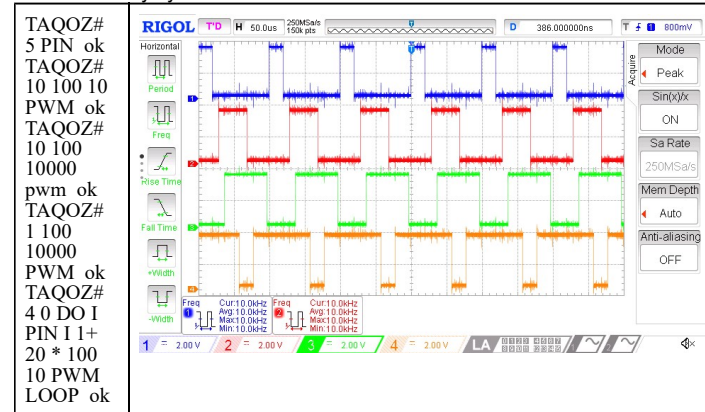
Make a new name for our dimmer as part of Forth and use it:

```
TAQOZ# : DIMMER ( value pin -- )    PIN 100 10 PWM ; ok
TAQOZ# 5 6 DIMMER ok
TAQOZ# 20 6 DIMMER ok
TAQOZ# 20 5 DIMMER ok
```

We could have elected to not specify a pin directly but leave it out like the other examples and perhaps that would be more consistent like this instead:

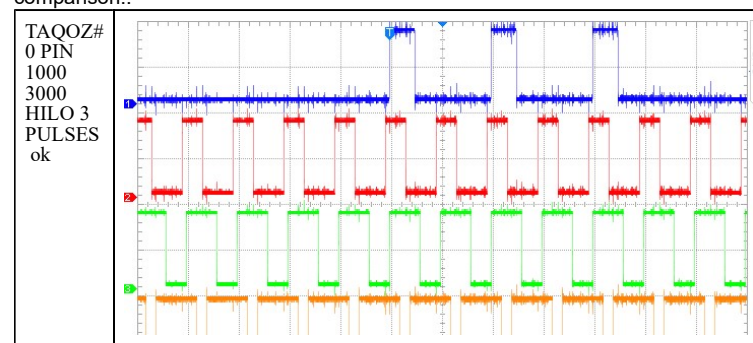
```
TAQOZ# : DIMMER ( value -- )    100 10 PWM ; ok
TAQOZ# 4 PIN 50 DIMMER ok
TAQOZ# 5 PIN 1 DIMMER ok
TAQOZ# 5 DIMMER ok
```

Lastly in this console capture we setup 4 pins to output 4 different rates from 20 to 80% duty cycle with the resultant waveforms



Another mode that is very useful is being able to generate an exact number pulses with controlled high and low times.

We can do it like this and show it alongside our previous PWM pulses for comparison.:



After specifying HILO we can play with 2 or 10 or 100 pulses etc.

NEW WORDS

WORD	STACK	DESCRIPTION
PWM	(on frame freq --)	Enable PWM that turns on for on cycles, off for frame-on, and freq as a prescaler (more=slower)
HILO	(high low --)	Specify High and Low times for subsequent PULSE or PULSES words
PULSES	(cnt --)	Generate cnt pulses according to specified high and low times.
PULSE		Sames as PULSES but only one pulse

SMARTPINS AS ASYNCHRONOUS SERIAL UARTS

One liners are handy but extending TAQOZ with your own words is easy too. Here I set pin 2 as a 20M baud UART transmit line, create a transmit string word so that I can pass the string and the pin number to it, and of course I test it.

```
TAQOZ# : TRANSMIT$ ( string pin -- ) PIN DUP LENS$ TXDAT ; ok
TAQOZ# 2 PIN 20 M TXD ok
TAQOZ# " HELLO WORLD" 2 TRANSMIT$ ok
```



(20M was the limit of the scope's decoder)

Exercise:
Create a new word that allows a string to be sent on any pin at any baudrate. (assume P2 clock of 180MHz)

Usage:
" Hello World" 115,200 8 SEND\$
" Parallax P2 Smart Pins are fast and fun! " 20,000,000 9 SEND\$
" Can any other micro do this?" 90,000,000 10 SEND\$

Solution: (drag mouse through hidden text in box to reveal and copy)
: SEND\$ (string baud pin --) PIN TXD DUP LENS\$ TXDAT ;

<WORK IN PROGRESS>
The [new section is here](#) while this section at the end of the TAQOZ document will be removed (just pending some checks on updates)

TAQOZ ROM GLOSSARY of WORDS

There are 432 words in TAQOZ ROM, for mere humans too much to memorise. So it's a good idea to keep this glossary by you when programming. The words are grouped under headings like Maths, Stack Operations to speed up searches.

The experienced Forth programmer will notice many words that are unique to TAQOZ. TAQOZ is not intended to meet any international standard; and for a good reason. The non-standard word set is a very close fit around the unique P2 processor and its special circuits. This results in a smaller, more useful

TAQOZ - Tachyon P2 Forth in ROM

Updated automatically every 5 minutes

The STACK column shows the effect each word has on the data stack
The DESCRIPTION column briefly describes what the word does
The CONSOLE EXAMPLES column(s) show the TAQOZ prompt and user input followed by the response and a final "ok" as indication of acceptance. It may be confusing therefore for those unfamiliar with Forth-like languages as to what constitutes user input and what constitutes the response. Rather than explaining all this **the user input is colored in** as a guide.

Sample word description

Here this row covers both the FOR and the NEXT word since they are used together to form a simple loop.

WORDS	STACK	DESCRIPTION	CONSOLE EXAMPLES
FOR NEXT	(cnt --) (--)	Simple counted loop FOR	TAQOZ# CRLF 3 FOR ." HELLO! " NEXT HELLO! HELLO! HELLO! ok

Data is referred to as bytes (8 bits), words (16 bits), longs (32 bits) and doubles (64 bits).

Boolean values of FALSE and TRUE are simply 0 or -1 (\$FFFF_FFFF) although in fact any non-zero value can be accepted as a boolean true but not necessarily suitable for bitwise operations such as AND. For instance **4 IF** and **8 IF** would both resolve as true however **4 8 AND IF** fails. Use 0<> to promote any non-zero to a full TRUE in such cases.

LIST OF DEBUGGING CONTROL KEY SHORTCUTS

ESC ESC ESC ESC ^C ^W ^D ^Q ^S ^B ^X ^Z ^?	Reset (works even if TAQOZ is not checking the console) Discard the current console line input Reset List WORDS in dictionary LSD Debugger in ROM (use ESC<CR> to return to TAQOZ) Print data stack Clear data stack Block memory dump of first 64kB Re-eXecute last line (very useful) Cold start Zap - wipe new code and dictionary - restore settings DEBUG List registers, current code an dictionary memory, and I/O
--	---

DEBUGGING

WORDS	STACK	DESCRIPTION	CONSOLE EXAMPLES
@WORDS	(-- adr)	Returns adr, the start of the dictionary	TAQOZ# @WORDS \$20 DUMP 0B3F9: 04 44 45 4D 4F 00 10 03 44 55 50 73 00 04 4F 56 'DEMO...DUPs.OV' 0B409: 45 52 77 00 04 53 57 41 50 7E 00 03 52 4F 54 82 'ERw..SWAP~.ROT' ok
WORDS	(--)	List the current dictionary of words ^W shortcut	
DUMP:		Use the following three words for byte, word, and long memory reads during a subsequent DUMP	pub SD DUMP: SDC@ SDW@ SD@ ; 0 \$100 SD DUMP
DUMP DUMPW DUMPL DUMPA DUMPAW	(adr bytes --)	Dump selected memory as:- bytes and ASCII words and ASCII longs and ASCII 64 ASCII characters wide (no hex) 128 ASCII characters	TAQOZ# \$FC000 \$20 DUMP --- 000F_C000: 00 08 80 FF 3F 00 0C FC 32 C8 06 F6 1F 04 DC FC '....?..2.....' 000F_C010: 40 7E 74 FD 01 CA A6 F0 1F CA 26 F4 00 CA 62 FD '@~t.....&...b.' ok3 TAQOZ# \$FC000 \$20 DUMPL --- 000F_C000: FF80_0800 FC0C_003F F606_C832 FCDC_041F '....?..2.....' 000F_C010: FD74_7E40 F0A6_CA01 F426_CA1F FD62_CA00 '@~t.....&...b.' ok

TAQOZ - Tachyon P2 Forth in ROM

Updated automatically every 5 minutes

		content	
COG LUT RAM SD SF	(--)	Modifies the next DUMP to use COG or LUT memory instead of the default HUB RAM. Resets to RAM after each DUMP	TAQOZ# 0 4 COG DUMP 00.0000: FDA0.0990 FD90.00BC 0000.0000 0000.0000 ok
.S	(--)	List the contents of the data stack Also accessible as the ^Q shortcut	TAQOZ# \$CAFEBABE DUP \$FFFF AND SWAP 16 >> .S DATA STACK x2 1 \$0000.CAFE 51966 2 \$0000.BABE 47806 ok
DEBUG	(--)	Dumps the various stacks, task registers, and quick dumps of sections of the current code and dictionary memory. ^? shortcut	
Isio	(--)	List the I/O, displays all 64 I/O pins and indicates what impedance the P2 senses on each pin	TAQOZ# Isio --- P:000000000011111111222222222333333333344444444455555555566 P:01234567890123456789012345678901234567890123456789012345678901 =:----- ok
LAP .LAP .ms CNT@ LAP@	(--) (--) (-- n) (-- n)	Time the code between each LAP and report results with .LAP Report only milliseconds CNT@ reads the 32-bit CNT register LAP@ is used internally by .LAP	TAQOZ# LAP 1000 FOR NEXT LAP .LAP --- 32146 cycles = 401.825us ok TAQOZ# LAP 1000 FOR NEXT LAP .ms --- 401.825us ok
RESET	(--)	Reset the P2 chip ^C or <break> shortcut	

THE STACKS

TAQOZ is based on Tachyon Forth which maintains a **data stack** in LUT memory in each cog but keeps a copy of the top four elements in fixed registers in cog memory for quick and efficient access. Therefore there are no (ugly) PICK and ROLL operators so as to encourage clean and efficient coding using a minimum of stack manipulation. The "top of stack" element or the last one pushed etc is referred to simply as "tos". The stack is a last in, first out store. Functions that require and pass parameters drawn from the top of the stack to usually no more than a depth of four elements.

TAQOZ has a **return stack** in which code return addresses are stored, so that words can be nested within words within words and so on to make a complete program.

In addition to the data and return stack, there is a dedicated **loop parameter stack** and a **branch stack** used internally by looping structures. Since the return stack is only used for return addresses we can then also access loops and parameters from any section of code that is not part of that loop.

the following letters mean
n - signed long (32 bit)
u - unsigned long (32 bit)
w - signed word (16 bit, sign extended to 32bits)
uw - unsigned word (16 bit, padded with leading 0s to 32 bits)
c - byte or char (8 bit, padded with leading 0s to 32 bits)
d - signed double (64 bit, stored as two longs on the stack, m.s. half nearest
tos)
ud - unsigned double (64 bit, stored as two longs on the stack, m.s. half
nearest tos)
adr - 20 bit address
str - the address of a zero delimited string
Other mnemonics are used where useful, but are generally self-explanatory

STACK OPERATIONS

WORDS	STACK	DESCRIPTION	CONSOLE EXAMPLES	
ISP	(--)	Initialise the data stack to empty Shortcut: ^S		
2DUP	(n1 n2 -- n1 n2 n1 n2) or (d -- d d)	Make a copy of a double at tos		
DUP OVER 3RD 4TH	(n -- n n) (n1 n2 -- n1 n2 n1) (n1 n2 n3 -- n1 n2 n3 n1) (n1 n2 n3 n4 -- n1 n2 n3 n4 n1)	Make a copy of the 1st, 2nd, 3rd, or 4th tos	TAQOZ# 1234 5678 9012 3456 .S DATA STACK x4 1 \$0000.0D80 3456 2 \$0000.2334 9012 3 \$0000.162E 5678 4 \$0000.04D2 1234 ok	TAQOZ# 4TH .S DATA STACK x5 1 \$0000.04D2 1234 2 \$0000.0D80 3456 3 \$0000.2334 9012 4 \$0000.162E 5678 5 \$0000.04D2 1234
?DUP	(n1 -- n1 n1) else (n -- n)	Duplicate the tos only if it is non- zero	TAQOZ# 1234 ?DUP 0 ?DUP .S DATA STACK x3 1 \$0000.0000 0 2 \$0000.04D2 1234 3 \$0000.04D2 1234 ok	
DEPTH	(-- n)	n = the current depth of the data stack (not including n)	TAQOZ# 1 2 3 ok	TAQOZ# DEPTH . 3 ok
DUPC@	(addr -- addr byte)	Duplicate the address and read a byte		
DROP 2DROP 3DROP	(n --) (n1 n2 --) or (d --) (n1 n2 n3 --)	Drop and discard the top elements of the data stack, either 1, 2, or 3 levels	TAQOZ# 1234 5678 9012 .S DATA STACK x3 1 \$0000.2334 9012 2 \$0000.162E 5678 3 \$0000.04D2 1234 ok	TAQOZ# 2DROP .S DATA STACK x1 1 \$0000.04D2 1234 ok
NIP	(n1 n2 -- n2)	Nip out the second element	TAQOZ# 1234 5678 9012 NIP .S DATA STACK x2 1 \$0000.2334 9012	

			5678 2 \$0000.04D2 1234 ok	1234 2 \$0000.162E 5678 ok
2SWAP	(n1 n2 n3 n4 -- n3 n4 n1 n2) or (d1 d2 -- d2 d1)	Swap the top two doubles		
ROT	(n1 n2 n3 -- n2 n3 n1)	Rotate the 3rd element to the top	TAQOZ# 1234 5678 9012 .S DATA STACK x3 1 \$0000.2334 9012 2 \$0000.162E 5678 3 \$0000.04D2 1234 ok	TAQOZ# ROT .S DATA STACK x3 1 \$0000.04D2 1234 2 \$0000.2334 9012 3 \$0000.162E 5678 ok
-ROT	(n1 n2 n3 -- n2 n1 n2)	Rotate the top element to the 3rd position (Reverse ROT)	TAQOZ# 1234 5678 9012 .S DATA STACK x3 1 \$0000.2334 9012 2 \$0000.162E 5678 3 \$0000.04D2 1234 ok	TAQOZ# -ROT .S DATA STACK x3 1 \$0000.162E 5678 2 \$0000.04D2 1234 3 \$0000.2334 9012 ok
>L L>	(n --) (-- n)	Push tos onto the loop stack Pop n from the loop stack		
>R R>	(n --) (-- n)	Push n onto the return stack Pop n from the return stack		

Note: The data stack is 4 levels deep in the cog and then implemented as a non-addressable LIFO stack in cog memory. TAQOZ words are optimized for these four fixed cog registers and to encourage efficient stack use no messy PICK and ROLL words are implemented. There are many words that also avoid pushing and popping the stack as this slows execution speed too. Try to factor words so that they use four or less parameters.

LOGIC (BITWISE)

WORDS	STACK	DESCRIPTION	CONSOLE EXAMPLES
AND ANDN OR XOR	(n1 n2 -- n3) (n1 n2 -- n3) (n1 n2 -- n3) (n1 n2 -- n3)	n3 = n1 and n2 n3 = n1 and not n2 n3 = n1 or n2 n3 = n1 xor n2	TAQOZ# 1011b 101b AND .BIN %1 ok TAQOZ# 1011b 101b NAND .BIN %1010 ok TAQOZ# %1000 %11 OR .BIN %1011 ok TAQOZ# %1010 %11 XOR .BIN %1001 ok
NOT	(n1 -- n2)	n2 = n1 all bits inverted	TAQOZ# %11 NOT .BIN %11111111111111111111111111111100 ok

SHIFTING

WORDS	STACK	DESCRIPTION	CONSOLE EXAMPLES
>> <<	(n1 cnt -- n2)	Shift n1 right or left by cnt bits	TAQOZ# \$F 2 >> . 3 ok TAQOZ# 3 2 << . 12 ok
ROL ROR	(n1 cnt -- n2) (n1 cnt -- n2)	n2 = rotate n1 left cnt times, with the ms bit rotating into the ls bit n2 = rotate n1 right cnt times, with the ls bit rotating into the ms bit	TAQOZ# \$F0000000 2 ROL HEX . \$C0000003 ok TAQOZ# \$15 3 ROR HEX . \$E0000001 ok

TAQOZ - Tachyon P2 Forth in ROM

Updated automatically every 5 minutes

2* 4* 8<< 9<<	(n1 -- n2)	Shift tos left by 1, 2, 8, or 9 **	TAQOZ# 3 8<<. 48 ok
--	-------------------------------	---------------------------------------	----------------------------------

** These instructions work with a single operand and return with a single operand. Because they do not push or pop they are useful, being both faster and more compact. For instance the word 8>> might simply be equivalent to 8 >> except it only needs a single wordcode and saves time by not having to push 8 and then pop it to discard afterwards since it is essentially a single PASM instruction.

CONVERSION AND MASKING

WORDS	STACK	DESCRIPTION	CONSOLE EXAMPLES
>9	(n -- 9bits)	9bits = n and \$1FF	
>B	(n -- byte)	byte = n and \$FF	
B>L	(a b c d -- dcba)	Merge four bytes into a long	
B>W	(a b -- word)	Merge two bytes into a word	
W>L	(word1 word2 -- long)	Merge words into a long (word1 word2 -- long)	
W>B	(word - - wordl wordh)	Split word into two bytes	
L>W	(long -- wordl wordh)	Split long into two words	
>N	(n -- nibble)	nibble = n and \$F (n -- nibble)	
>W	(n -- word)	word = n and \$FFFF	
BITS	(n1 cnt -- n2)	Extract l.s. cnt bits from n1	TAQOZ# \$F0F0F0F0 5 BITS . 32 ok
L>S	(n -- lsb9 h)	Specialized operation for file system addresses	
REV	(n1 -- n2)	Reverse bits 0..32 -> 32..0 (n1 -- n2) Very useful for array address twiddling in fast fourier transforms	TAQOZ# 5 REV . 167772160 ok
> 	(bitmask -- bitpos)	Encode - bitpos = the position of the msb set in bitmask	TAQOZ# %0100 > . 2 ok TAQOZ# 64 > . 6 ok
 <	(bitpos -- bitmask)	Decode - bitmask = long with one bit set at bitpos position	TAQOZ# 3 < .BIN %1000 ok TAQOZ# 6 < .BIN %1000000 ok
S>#	(str -- u digits false)	Converts string at str to a number u and digit count digits, else returns false on fail	

MATHS

WORDS	STACK	DESCRIPTION	CONSOLE EXAMPLES
*/	(n1 n2 n3 -- res)	Multiply n1 by n2 and divide the 64-bit product by n3 for a 32- bit result	TAQOZ# 355 100000000 113 */ . --- 314159292 ok
+ - * /	(n1 n2 -- n3)	Add, subtract, multiply, divide	TAQOZ# 2 3 + . 5 ok TAQOZ# 12 -4 / . -3 ok

TAQOZ - Tachyon P2 Forth in ROM

Updated automatically every 5 minutes

MAX MIN	(u1 u2 -- u3)	Leave the maximum, minimum of u1 and u2, all unsigned	TAQOZ# 4 5 MAX . 5 ok TAQOZ# 33 99 MIN . 33 ok
MAXS MINS	(n1 n2 -- n3)	Leave the maximum, minimum of n1 and n2, all signed	
NEGATE -NEGATE ?NEGATE	(n1 -- n2) (n1 n2 -- n3) (n1 flag -- n2)	n2 = -n1 n3 = -n1 if n2 is negative, else n3=n1 n2 = -n1 if flag is true, else n2=n1	TAQOZ# 45 NEGATE . -45 ok TAQOZ# 54 -2 -NEGATE . -54 ok TAQOZ# -2 TRUE ? NEGATE . 2 ok
GETRND	(-- n)	Returns pseudorandom number *	
RND	(-- n)	Generate n, a pseudo-random long enhanced with the system counter	
SQRT	(u1 -- u2)	u2 = unsigned square root of u1	TAQOZ# 9 3 3 * * SQRT . 9 ok
UM*	(u1 u2 -- ud1)	Unsigned u1 * u2, resulting in unsigned 64 bit ud1	
UM//	(ud1 u2 -- rem quot)	Unsigned 64 bit ud1 divided by long u2 resulting in rem and quotient unsigned longs	
U/ U//	(u1 u2 -- u3) (u1 u2 -- rem quot)	Divide u1 by u2, all unsigned Divide u1 by u2, all unsigned	
W*	(w1 w2 -- n)	n = w1*w2 (fast single PASM instruction)	
1+ 2+ 4+	(n1 -- n2)	Increment the tos element by 1, 2, or 4 *	TAQOZ# 1234 2+ . 1236 ok
1- 2-	(n1 -- n2)	Decrement the tos element 1 or 2 *	TAQOZ# 1234 2- . 1232 ok

* These instructions work with a single operand and return with a single operand. Because they do not push or pop they are useful, both faster and more compact. For instance the word 1+ might simply be equivalent to 1 + except it only needs a single wordcode and saves time by not having to push 1 and then pop it to discard afterwards since it is essentially a single PASM instruction.

MODIFIERS

WORDS	STACK	DESCRIPTION	CONSOLE EXAMPLES
KB MB M	n1 -- n2 n1 -- n2 n1 -- n2	n2 = n1*1,024 n2 = n1*1,048,576 n2 = n1*1,000,000	TAQOZ# 128 KB . --- 131072 ok

STRINGS

WORDS	STACK	DESCRIPTION	CONSOLE EXAMPLES
[“]	(--)	During compilation, displays the following string in the input stream until a terminating “	
“	(-- str1)	Compile a string in the input stream up to the trailing " and leave the address of the null terminated string on the stack.	TAQOZ# " hello world" \$10 DUMP --- 01002: 68 65 6C 6C 6F 20 77 6F 72 6C 64 00 10 F8 44 D8 'hello world...D.' ok

Updated automatically every 5 minutes

Note: All TAQOZ strings are 0 delimited

WORDS	STACK	DESCRIPTION	CONSOLE EXAMPLES	
byte <variable name> word <variable name> long <variable name>	(--) (--) (--)	Define new byte variable Define new word variable Define new long variable	TAQOZ# --- defining vars TAQOZ# byte MYBYTE ok TAQOZ# word TIMEOUT ok TAQOZ# long SAMPLE ok	TAQOZ# --- writing to a long var. TAQOZ# 2000 SAMPLE ! ok TAQOZ# --- reading a long var. TAQOZ# SAMPLE @ . 2000 ok
bytes <variable name> words <variable name> longs <variable name>	(n --) (n --) (n --)	Define new byte array variable Define new word array variable Define new long array variable All these create an array of size n bytes, words or longs	TAQOZ# 4 longs BUFFER ok	TAQOZ# pub INITBUFFER (--) TAQOZ# --- set all entries to 100 TAQOZ# 4 0 DO TAQOZ# 100 BUFFER I 4* +! TAQOZ# LOOP ; ok
org res	(adr --) (n --)	Set data pointer used for defining variables to hub memory adr (for example to reclaim data space when reloading source code) Takes next word in input stream as a ... and reserves n bytes of hub memory	TAQOZ# IFDEF *MANDELBROT* TAQOZ# Cx org TAQOZ# --- Cx is the 1st variable defined in the code below this TAQOZ# FORGET *MANDELBROT* TAQOZ# } ok TAQOZ# 8 res fnam ok	

Note 3: None of these words initialise the variable, they just allot the required memory.

WORDS	STACK	DESCRIPTION	CONSOLE EXAMPLES
<code>:=</code>	<code>(n --)</code>	Create a long constant using the name next in the input stream	TAQOZ# 1 := RED ok TAQOZ# RED . 1 ok TAQOZ# 2 ' RED ==! Ok TAQOZ# RED . 2 ok

TAQOZ - Tachyon P2 Forth in ROM

Updated automatically every 5 minutes

	(c --)	Compile byte c into code memory	
	(w --)	Compile word w into code memory	
,	(n --)	Compile long n into code memory	

PREDEFINED TASK VARIABLES

Each cog may have its own set of variables that are offset from a base address (that address is obtained by using REG - see below)
This is so that any cog running TAQOZ may have different I/O devices selected etc.

WORDS	STACK	DESCRIPTION	CONSOLE EXAMPLES
REG	(index -- adr)	Find the address of register 'index'	
delim	(-- adr)	Word delimiter (normally space) plus backup byte with delimiter detected (SP,TAB,CR etc)	
names	(-- adr)	Points to the start of the latest name field in the dictionary (builds down)	
uemit	(-- adr)	Vector that points to cfa of current EMIT routine (0=console=(EMIT))	
ukey	(-- adr)	Vector that points to cfa of current KEY routine (0=console=(KEY))	

Note: There are more registers available, the above are all that are defined in TAQOZ ROM. The user may create more words to access the extra registers with the aid of the ROM assembly code listing. TAQOZ RELOADED includes more words for register access too.

PREDEFINED CONSTANTS

WORDS	STACK	DESCRIPTION	CONSOLE EXAMPLES
ON TRUE -1 OFF FALSE	(-- n) (-- n)	n = -1 n = 0	TAQOZ# ON OFF .S DATA STACK x2 1 \$0000.0000 0 2 \$FFFF.FFFF -1 ok
CLKHZ	(-- n)	n = 20,000,000	

Note: Constants can be changed using ==!

DEFINING NEW WORDS

WORDS	STACK	DESCRIPTION	CONSOLE EXAMPLES	
[]		flag that we have entered a definition end definition and lock allocated bytes		
pub :	(--)	Create new word as public	TAQOZ# pub HITHERE ." Hello World" ; ok TAQOZ# : HITHERE2 ." Hello World" ; ok	TAQOZ# HITHERE Hello World ok
pri	(--)	Create a new word that is marked private so a RECLAIM can remove them from the dictionary when	TAQOZ# pri HIAGAIN ." Hello the World" ; ok	TAQOZ# HIAGAIN Hello the World ok TAQOZ# RECLAIM ok TAQOZ# HIAGAIN ?? hiagain not found

TAQOZ - Tachyon P2 Forth in ROM

Updated automatically every 5 minutes

		dictionary uncluttered from unnecessary detail		
pre	(--)	Create a new word that has the "immediate" attribute set so that it executes immediately at compile time (words like IF ELSE THEN etc)	TAQOZ# pre <new word name> more TAQOZ words ; ok	
;	(--)	Finalise a new word	See examples above	
ALLOT	(bytes --)	Allot n bytes of code memory - advances "here"		
CREATE\$ CREATE	(str --) (--)	Creates a new word in the dictionary using the string at address str. If string is empty no new word is created Creates a new word in the dictionary using the name that follows in the input stream		
CFA	(nfa -- cfa)			
CPA	(nfa -- cpa)	The CPA is the address of the word code stored in the name field header that points to the code to execute		
TAQOZ END	(--) (--)	Marks the start of a block of source code to be compiled in block mode Marks end of block load mode		

CONDITIONAL DEFINING

WORDS	STACK	DESCRIPTION	CONSOLE EXAMPLES
IFDEF IFNDEF	(--)	If the next name in the input stream is already DEFINED then process all input stream that follows up to the terminating curly brace } If next name in the input stream is NOT DEFINED then process all source between here and the matching curly brace }	TAQOZ# IFDEF *BREAKOUT* TAQOZ# bk org TAQOZ# FORGET *BREAKOUT* TAQOZ# } ok

DICTIONARY

WORDS	STACK	DESCRIPTION	CONSOLE EXAMPLES	
HERE	(-- adr)	Point to the next compilation location		

TAQOZ - Tachyon P2 Forth in ROM

Updated automatically every 5 minutes

@NAMES	(-- namadr)	Point to the latest entry in the names dictionary		
FORGET	(--)	Takes next word in input stream, and forgets all names in the dictionary from this name onwards - from most recent instance of the name		
SEARCH	(cstr -- nfaptr)	Search the dictionaries for cstr which points to a counted word string constructed as count+string+null		

COMPARISON

WORDS		DESCRIPTION	CONSOLE EXAMPLES
0= 0<> 0<	(n -- flag)	Compare tos with zero	TAQOZ# 0 0= . -1 ok TAQOZ# 0 0<> . 0 ok
= <> < > U< U> <= =>	(n1 n2 -- flag) (u1 u2 -- flag) (n1 n2 -- flag)	Compare top two stack values Compare two unsigned longs Compare top two stack values	TAQOZ# 1 1= . -1 ok TAQOZ# 1 1<> . 0 ok
WITHIN	(val min -- max -- flag)	Return with flag true if val is within min and max (inclusive, not ANSI)	TAQOZ# 3 1 5 WITHIN . -1 ok TAQOZ# 3 3 7 WITHIN . -1 ok TAQOZ# 6 3 7 WITHIN . 0 ok

BRANCHING

WORDS	STACK	DESCRIPTION	CONSOLE EXAMPLES	
IF ELSE THEN	(cond --)	Conditional execution IF		
BEGIN AGAIN UNTIL WHILE REPEAT	(cond --) (cond --)	Start a repeated conditional loop Repeat forever Repeat until condition false Repeat while condition true End a WHILE loop	BEGIN xxx AGAIN BEGIN xxx cond UNTIL BEGIN xxx cond WHILE yyy REPEAT	
GOTO				
EXIT	(--)	Exit now		
0EXIT ? EXIT	(cond --)	Exit on 0 / false, or on true		

CASE

WORDS	STACK	DESCRIPTION	CONSOLE EXAMPLES	
BREAK	(--)	Stop executing the CASE code immediately and execute the code that follows	TAQOZ# pub CASETEST (val --) TAQOZ# --- print 'val' as a word TAQOZ# SWITCH TAGOZ# 1 CASE ." one" BREAK TAQOZ# 2 CASE ." two" BREAK TAQOZ# 3 CASE ." three"	
CASE	(val1 --)	Execute the following code up to BREAK if val1 = SWITCH val2		

TAQOZ - Tachyon P2 Forth in ROM

Updated automatically every 5 minutes

		SWITCH word		
CASE=	(val -- flag)	flag set true if val = value stored by the last SWITCH word		
CASE>	(from to -- flag)	flag set true if val stored by the last SWITCH word is within range 'from' - 'to', inclusive		

VECTORED EXECUTION

WORDS	STACK	DESCRIPTION	CONSOLE EXAMPLES
AUTO			
'	(-- adr false)	Takes the next word in the input stream, looks it up in the dictionary and returns the execution adr of the word else if not found returns false	
CALL			
JUMP	(adr --)	jump to address on top of the data stack	
NOP			

LOOPING

WORDS	STACK	DESCRIPTION	CONSOLE EXAMPLES
DO ADO LOOP +LOOP	(limit start --) (start cnt --) (--) (--)	Execute code between DO LOOP until the index equals the limit.	TAQOZ# 8 0 DO I. LOOP 01234567 ok TAQOZ# 0 8 ADO I. LOOP 01234567 ok TAQOZ# 0 8 ADO I. 2 +LOOP 0246 ok
FOR NEXT	(cnt --) (--)	Simple counted loop FOR	TAQOZ# CRLF 3 FOR ." HELLO!" NEXT HELLO! HELLO! HELLO! ok
LEAVE	(--)	Set the index to limit-1 so that it will LEAVE the loop at the next LOOP or +LOOP executed	
?NEXT	(flag --)	Exit FOR NEXT loop early if flag is true (non-zero)	
I J	(-- n) (-- n)	I returns the current loop index J returns the index for the next outer loop.	
I+	(n -- n+1)	Fast loop index e.g. to index through a table or array	

TIMING

WORDS	STACK	DESCRIPTION	CONSOLE EXAMPLES
us ms s	(n --)	Wait for microseconds, milliseconds, or seconds	TAQOZ# CNT@ 100 ms CNT@ SWAP - . 8000552 ok
CNT@ LAP .LAP LAP@		Timing code between each LAP and report results with .LAP. CNT@ reads the 32-bit	TAQOZ# LAP 1234 5678 * LAP .LAP 242 cycles = 3.25us ok

MEMORY			
WORDS	STACK	DESCRIPTION	CONSOLE EXAMPLES
ALIGN	(adr1 n -- adr2)	adr2 = adr1 realigned on an n'th byte boundary	TAQOZ# 0 4 ALIGN . 0 ok TAQOZ# 4 4 ALIGN . 4 ok TAQOZ# 1 4 ALIGN . 4 ok TAQOZ# 5 4 ALIGN . 8 ok
ERASE FILL	(adr cnt --) (adr cnt c --)	Erase hub memory starting at adr for cnt bytes, filling with byte 0 Fill hub memory started adr for cnt bytes, with c byte	TAQOZ# \$4.0000 \$10 SFA FILL . ok TAQOZ# \$4.0000 \$20 DUMP 04.0000: FA FA FA FA FA FA FA FA FA FA FA FA FA FA FA FA FA 04.0010: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 ok
CMOVE <CMOVE	(src dst cnt --) (src dst cnt --)	Move hub memory bytes starting from source to destination by cnt bytes Same as CMOVE, in reverse address order (Use one or the other when the two blocks overlap each other)	TAQOZ# @NAMES \$4.0000 \$20 CMOVE . ok TAQOZ# \$4.0000 \$20 DUMP 04.0000: 03 54 52 49 80 86 1F 03 53 41 57 80 80 1F 03 44TRI....SAW....D 04.0010: 55 50 80 6B 00 04 32 44 55 50 80 6D 00 04 4F 56UP.k..2DUP.m..OV ok
C@ W@ @	(adr -- c) (adr -- w) (adr -- n)	Fetch a byte, word, or long from hub memory	TAQOZ# @NAMES \$10 DUMP 00.CFF2: 03 54 52 49 80 86 1F 03 53 41 57 80 80 1F 03 44TRI....SAW....D ok TAQOZ# @NAMES C@ .BYTE 03 ok
C@++	(adr1 -- n adr2)	Fetch a byte from hub memory at adr1 and increment the address to adr2 leaving it tos	TAQOZ# \$1000 C@++ .S DATA STACK x2 1 \$0000.005C 92 2 \$0000.1001 4097 ok
C! W! !	(c adr --) (w adr --) (n adr --)	Store a byte, word, or long to hub memory	TAQOZ# \$DEADBEEF \$F000 ! ok TAQOZ# \$F000 \$10 DUMP 00.F000: EF BE AD DE 00 00 00 00 00 00 00 00 00 00 00 ok
C+! W+! +!	(c adr --) (w adr --) (n adr --)	Add parameter directly to hub memory location adr	TAQOZ# \$600 \$F000 +! ok TAQOZ# \$F000 \$10 DUMP 00.F000: EF C4 AD DE 00 00 00 00 00 00 00 00 00 00 00 ok
COG@ COG! LUT@ LUT!	(adr -- n) (n adr --) (adr -- n) (n adr --)	Read long at COG memory adr Write long to COG memory adr Read long at LUT memory adr Write long to LUT memory adr	TAQOZ# 0 COG@ .LONG FDA0.0990 ok TAQOZ# 0 LUT@ .LONG 0000.04D2 ok TAQOZ# \$CAFEBABE 2 COG! 2 COG@ .LONG CAFE.BABE ok
C++ C-- W++ W-- ++ --	(adr --) (adr --) (adr --) (adr --) (adr --) (adr --)	Increment byte at adr hub memory Decrement byte at adr hub memory Increment word at adr hub memory Decrement word at adr hub memory	

W~ W~~ ~ ~~	(adr --) (adr --) (adr --) (adr --)	Byte set \$FF at hub memory adr Word set 0 at hub memory adr Word set \$FF at hub memory adr Long set 0 at hub memory adr Long set \$FF at hub memory adr	
DATA?	(addr cnt -- flag)	Test for non-zero data starting at memory addr for cnt longs	
BIT! SET CLR SET?			

SERIAL FLASH

WORDS	STACK	DESCRIPTION	CONSOLE EXAMPLES
BACKUP	(--)	BACKUP the TAQOZ system with any added user words into the last 64KB of 1MB of Flash	
RESTORE	(--)	Restore the backup after reset	
.SF	(--)	Displays flash memory manufacturers' I.D. serial number etc. Useful for checking communication with flash is working	TAQOZ# .SF \$EF70_1800 \$0F0B_2826 \$E468_5CF4 ok
SF			
SF?			
SF@	(adr -- n)	Returns long from flash address adr	
SFC@	(adr -- c)	Returns byte from flash address adr	
SFW@	(adr -- w)	Returns word from flash address adr	
SFER4 SFER32 SFER64	(adr --)		
SFERASE	(--)		
SFINS			
SFPINS	(-- n)	Returns constant n = \$3d3a3b3c , the four pin numbers of the Flash memory interface	
SFJID	(-- n)	Read serial Flash Jedec ID	
SFRDS			
SFSID		Read serial Flash serial number	
SFWD			
SFWE			

SD CARD (FAT32)			
WORDS	STACK	DESCRIPTION	CONSOLE EXAMPLES
@BOOT	(sector str --)		
@FAT	(fat# -- sector)		
@ROOT			
!SD	(-- ocr false)	Initialise the SD card in SPI mode and return with the OCR, else false if an error occurs	
!SX			
ACMD	(data acmd -- res)		
cid			
CMD	(data cmd -- res)		
DIR	(--)	lists info about the card, FAT32 and directory entries	
fat	(-- n)	Constant n = \$FF00 the start location of the file allocation table	
FGET			
FLOAD		loads a TAQOZ source file	
FLUSH	(force --)		
FOPEN	(addr --)	Opens file by address visible in listing from DIR	TAQOZ# \$A11E00 FOPEN --- ok TAQOZ# 0 100 SD DUMP --- 00000: 66 6C 66 32 61 24 20 36 20 34 20 36 20 2D 31 20 'ñf2a\$ 6 4 6 -1 ' 00010: 34 0A 33 78 35 20 66 6F 6E 74 20 62 79 20 52 69 '4.3x5 font by Ri' 00020: 63 68 61 72 64 20 4B 69 72 6B 20 28 72 61 6B 40 'chard Kirk (rak@' 00030: 63 72 6F 73 66 69 65 6C 64 2E 63 6F 2E 75 6B 29 'crosfield.co.uk' 00040: 2E 0A 50 6F 72 74 65 64 20 74 6F 20 66 69 67 6C '..Ported to figl' 00050: 65 74 2C 20 61 6E 64 20 73 6C 69 67 68 74 6C 79 'et, and slightly' 00060: 20 63 68 61 6E 67 65 64 20 28 77 69 74 68 6F 75 ' changed (withou' ok
FREAD	(sdsr hubdst bytes --)		
FWRITE	(hubsr sddst bytes --)		
MOUNT	(--)	Mounts SD card and reports type, s/n, formatted capacity	TAQOZ# MOUNT.SDSC64G 6269_8201 P2 CARD 32k 60,890M ok

		card interface	
SD@ SD!	(xaddr -- long) (long xaddr --)		
SDC@	(xaddr -- byte)		
SDW@	(xaddr -- word)		
SD?	(-- flag)		
SDADR	(xaddr -- addr)		
SDRD	(sector dst --)		
SDRDS	(sector dst cnt -- crc false)		
SDWR	(src sect -- flag)	Write from src to xdst in the SD	
SDWRS	(hubsrc sdadr cnt --)		
SECTOR	(sect -- sdbuf)		

DEFAULT RADIX WORDS

WORDS	STACK	DESCRIPTION	CONSOLE EXAMPLES
HEX DECIMAL BINARY	(--)	Switch the number base to hexadecimal Switch the number base to decimal Switch the number base to binary	TAQOZ# 23 DUP . 23 ok TAQOZ# HEX ok TAQOZ\$. AC ok

Note: The TAQOZ prompt suffix changes to show the default radix. \$=hex
#=decimal %=binary

NUMBER I/O

WORDS	STACK	DESCRIPTION	CONSOLE EXAMPLES
. PRINT U.	(n --) (n --) (u --)	Display a signed number n Display a signed number n Display unsigned number u	
<# # #S HOLD #>	(c --) (-- str)	Start a new formatted number string Insert the next digit of the number being displayed into the formatted number string Insert all remaining significant digits of the number Insert the character on the stack into the formatted string Terminate the formatted number leaving the address of the formatted number string at tos, ready for display	TAQOZ# pub TODOLLARS (d --) TAQOZ# <# # # 46 HOLD #S 36 HOLD #> ; ok TAQOZ# 1234. TODOLLARS PRINT\$ \$12.34 ok

TAQOZ - Tachyon P2 Forth in ROM

Updated automatically every 5 minutes

.AS"	(n --)	Display n with format as per string spec: # Convert one digit (default is decimal) ~ Toggle leading zero suppression \ pad leading zeros with spaces \$ Hexadecimal * Convert all remaining digits 4 Convert 4 digits Terminated with "	TAQOZ# 32767 .AS" \$ ####" \$EFFF ok
.B .BYTE	(n --)	Display n as two digit hex byte 00-FF	TAQOZ# 253 .B DF ok
.BIN	(n --)	Display n in binary with leading %	TAQOZ# 33 .BIN %100001 ok
.DEC	(n --)	Display n in decimal with at least a single digit	
.DECL	(n --)	Display n as decimal "###,###,###,####"	
.DEC4	(n --)	Display n as decimal "####" (n --)	
.H	(n --)	Display n as hex nibble 0-F	
.L .LONG	(n --)	Display n as eight digit hex 00000000 - FFFFFFFF	
.W .WORD	(n --)	Display n as four digit hex 0000-FFFF	

CHARACTER I/O

WORDS	Stack	DESCRIPTION	CONSOLE EXAMPLES	
."	(--)	Print text in input stream until terminating "		
CONEMIT EMIT CLS CRLF CR SPACE SPACES EMITS		Output to the terminal		
CONKEY KEY WKEY	(-- c false)	Return with the next character in the console buffer or else a flag = false (0)		
CTYPE	(str cnt --)	Display cnt chars, starting at address str		
KEY!	(c --)	Force a character as the next key read in the input stream		
CON				
PRINT\$	(str --)	Display string at address str	TAQOZ# 16 BYTES mystring ok TAQOZ# " FRED" mystring \$! ok TAQOZ# mystring PRINT\$ FRED ok	
<D>				

TAQOZ - Tachyon P2 Forth in ROM

Updated automatically every 5 minutes

		respond to the 'backspace' char	would be other code within the loop, checking for some change in state
TERM		Main terminal console	

I/O PORTS

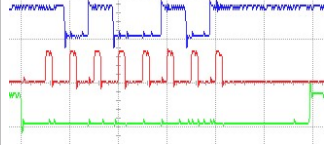
WORDS	STACK	DESCRIPTION	CONSOLE EXAMPLES
HIGH LOW FLOAT	(pin --)	Set the direction and output of a pin	TAQOZ# 4 HIGH 4 PIN@ . -1 ok TAQOZ# 4 LOW 4 PIN@ . 0 ok
H L F R T	(--) (--) (--) (-- flag) (--)	Set a preselected pin to a high output Set a preselected pin to a low output Set a preselected pin to a floating state(high impedance) Set a preselected pin to input, flag = input state	TAQOZ# 4 PIN ok TAQOZ# L R . 0 ok TAQOZ# H R . -1 ok

SMARTPINS

WORDS	STACK	DESCRIPTION	CONSOLE EXAMPLES
WRPIN WXPIN WYPIN RDPIN RQPIN AKPIN	(dst --) (dst --) (dst --) (-- res) (-- res) (--)	Same as assembler - write dst to mode register of smart pin S[5:0], acknowledge smart pin. Write dst to parameter "X" of smart pin S[5:0], acknowledge smart pin. Set smart pin S/# parameter Y to dst res = smart pin S/#, ack Res = smart pin S/#, don't ack /#, acknowledge smart pin	
COM NONE		direct terminal output to a smartpin (after init)	
WRACK	(n --)	Write smartpin data, wait for empty then acknowledge	
WAITPIN TXDAT DUTY NCO	(buf cnt --) (val --) (n --)	Write buffer direct to WYPIN Set Numerically Controlled Oscillator duty cycle Start Numerically Controlled Oscillator, n sets relationship to system clock - see P2 manual	TAQOZ# 48 PIN \$8000 0000 NCO ok TAQOZ# --- pin 48 emits ½ system clock ok
HILO	(high low --)	Specify High and Low times for subsequent PULSE or PULSES words	
HZ KHZ MHZ BLINK	(freq --) (--)	Set PIN to NCO mode and output the frequency BLINK sets an output to 2 Hz - for LED flashing	TAQOZ# 4 PIN 10 MHZ ok TAQOZ# 5 PIN BLINK ok

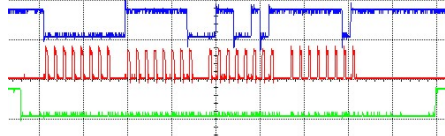
@PIN	(-- pin)	use, this setting is remembered until another pin is selected Read setting back	@PIN . 5 ok
PULSE			
PULSES	(cnt --)	Generate cnt pulses according to specified high and low times	
PW	(width --)		
PWM SAW	(duty frame div --)	Set pin to PWM triangle mode using duty, frame, and divider parameters. Use SAW instead for sawtooth.	TAQOZ# 4 PIN 1000 4000 1 PWM ok
SEROUT			
SETDACS	(n --)	DAC3 set to bits 31-24, DAC2 set to bits 23-16, DAC1 set to bits 15-8, DAC0 set to bits 7-0 of n	
BAUD	(baud mode --)		
TXD	(mode baud --)	Setup a pin as an asynchronous transmit using the supplied parameter as a baud rate	TAQOZ# 34 PIN 8 BIT 40 M TXD @NAMES \$10 TXDAT ok
RXD	(mode baud --)	Setup a pin as an asynchronous receive using the supplied parameter as a baud rate	
BIT	(n --)	Specify the data bits (1..32) to be used whenever a pin is switched to RXD or TXD modes. If this is not specified it will default to 8	TAQOZ# 8 BIT 115200 TXD ok

SPI
Serial Peripheral Interface is implemented as cog instructions that handle half-duplex read or write in increments of 8-bits. Once the SPI pins have been set then any read or write will automatically enable the Chip Enable and only requires a SPICE to turn it back off if required. Sometimes all that is required here is to SPICE just before starting a new command to reset the slave.

WORDS	STACK	DESCRIPTION	CONSOLE EXAMPLES
SPIWB SPIWW SPIWC	(n1 -- n1)	Write Byte or Word or SD Command to SPI pins. 200ns/div example sending \$A5 then SPICE 	TAQOZ# \$12345678 SPIWB .S DATA STACK x1 1 \$0000.0078 120 ok
SPIWL			
SPIWM			
SPIRD	(n1 -- n2)	Read 8-bits left into n1 so that n2 = n1<<8+new. Four successive SPIRDs will receive 32-bits	

TAQOZ - Tachyon P2 Forth in ROM

Updated automatically every 5 minutes

SPIRX SPITX SPITXE	(adr cnt --)	Send or receive between memory and SPI 500ns/div example writing 4 bytes from memory 	TAQOZ# &32.33.34.35 SPIPINS ok TAQOZ# 0 4 SPITXE SPICE ok
---	-----------------------	---	--

COG/HUB

WORDS	STACK	DESCRIPTION	CONSOLE EXAMPLES
COGID COGINIT COGSTOP COGATN	(-- cog#) (adr cog# --) (cog# --) (maskw --)	Return current cog# Same as COGINIT in PASM - also saves information in cog TASK block at adr Stop cog# Strobe the "attention" event flag for all cogs whose corresponding bits are high in maskw (bit 0 - 8 for P2)	
NEWCOG	(n --)	An idling instance of TAQOZ is loaded and run in cog#n	
POLLATN SETEDG POLLEDG	(-- flag) (edge pin --) (-- flag)	flag = ATN attention event flag, ATN event flag then cleared	
HUBSET WP WE REBOOT	(n --)	HUBSET - configure various global circuits Write Protect ROM Write Enable ROM	
TASK	(cog# -- adr)	Index the cog's task variable and return with its address	
WAITCNT	(--)	Continue from last count (must be called before target is reached)	
WAITX	(delta --)	Calculate and set the cnt delta and waitcnt	

COMMENTS

WORDS	DESCRIPTION	CONSOLE EXAMPLES
{ <multiline comment the compiler ignores> }	Ignores all text up to the matching curly brace }	
--- <rest of line ignored> \ <rest of line ignored> // <rest of line ignored>	Ignores the rest of the line Ignores the rest of the line, do not echo Ignores the rest of the line	
(<words to ignore>)	Ignores characters until closing brace) Use on one line only, else use { } multiline	

Words yet to find a home in a table - you could help if you know where they go!:-

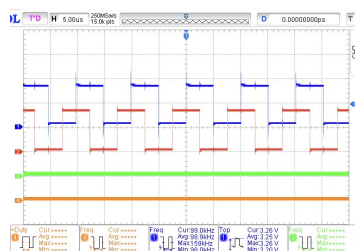
TAQOZ - Tachyon P2 Forth in ROM

Updated automatically every 5 minutes

new line to display error			
char			
[C]		force compilation of the next word	
[W]		append this wordcode to next free code location + append EXIT (without counting)	
ASM			
VAR			
AUTO		autorun	
WSLED		something to do with driving an intelligent LED WS2812	
DATCON			
GRAB		IMMEDIATE --- executes preceding code to make it available for any immediate words following	
CLKDIV RCSLOW			

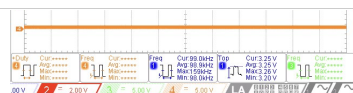
Booting

- **GETTING THE TAQOZ ROM PROMPT UP**
- **SD-Boot FAT32:** The root directory is searched for file "_BOOT_P2.BIX", and if not found then the root directory is searched for file "_BOOT_P2.BIY" (upper case is required). If the file is found, then all files' data is loaded into HUB \$0. Maximum file size is 512KB-16KB = 496KB. Once loaded into HUB, the first 496 longs will be copied to COG \$0 and then a JMP #\$0 will be executed. Note: The files' data must be in contiguous sectors.
- **SD-Boot:** The SD boot will check for valid P2 Boot Data on the MBR sector (sector 0), and if not found it will try the VOL sector (provided the SD Card is in FAT32 format).
 - 1. Valid Boot Data is "Prop" at offset \$17C on the MBR/VOL sector...The sector (512 bytes) will be read into Hub \$0, followed by a JMP \$080. Only code in sector byte offset \$080-\$17F is considered valid but this program may know that other areas are also valid.
 - 2. Valid Boot Data is "ProP" at offset \$17C on the MBR/VOL sector...The long at offset \$174 is used as the sector start address, and the long at \$178 is used as the maximum filesize (in bytes) is 512B-16KB = 496KB for the data to be loaded into Hub, followed by a JMP \$000.
 - These two methods also cover the case where the SD Card is not formatted as FAT32 (eg exFAT). The boot sector(s) MBR and/or VOL must be specifically written with a program such as HxD on Windows 10.



TAQOZ - Tachyon P2 Forth in ROM

Updated automatically every 5 minutes



If anyone is interested, this is the way I would do it in TAQOZ in one line of code and hit enter.

```
48 PIN 12000 6000 HILO 50 LOW 16 PULSES WAITPIN 3 PULSES  
50 HIGH
```

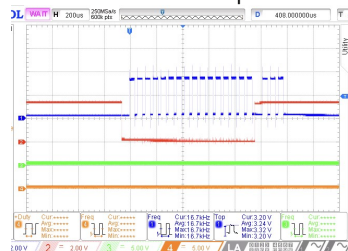
My clock is 300MHz so 40us= 12000 and I specify P48 as the target smartpin.

HILO takes high and low count and sets up the Smartpin.

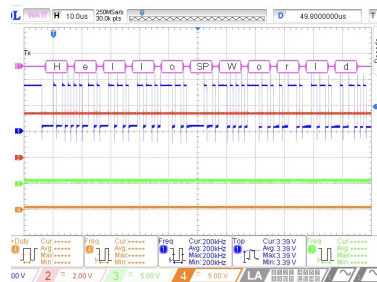
LOW drives P50 low.

PULSES writes wypin with that value (just an alias)

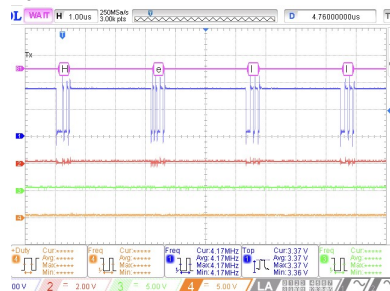
WAITPIN waits for smartpin ack



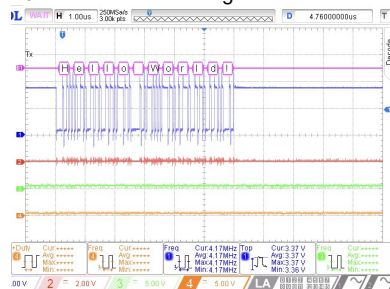
1Mbd hello world



20Mbd hello world



20Mbd hello world using txdat



TAQOZ# 52 pin 825 mV --- ok

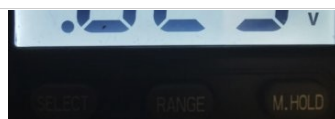
 Published using Google Docs

[Report abuse](#)

[Learn more](#)

TAQOZ - Tachyon P2 Forth in ROM

Updated automatically every 5 minutes



Meter reading